Proyecto Integrador

Etapa 2: Desarrollo de una tienda móvil con Flutter e integración con Laravel.

Desarrollar una aplicación móvil de comercio electrónico utilizando Flutter, que permita consultar el catálogo de productos y realizar el proceso de compra, desde la selección de productos hasta el registro del pedido y su pago.

La aplicación deberá integrarse con el backend Laravel desarrollado en la etapa 01 mediante peticiones HTTP asincrónicas e intercambio de datos en formato JSON. Flutter será responsable de la interfaz de la tienda.

El backend conservará la responsabilidad sobre la lógica de negocio, la validación, la autenticación, la autorización y la persistencia. Se deberán implementar o adaptar los endpoints necesarios, reutilizando los modelos y funcionalidades desarrollados en la etapa anterior.

No es necesario trasladar a Flutter los módulos administrativos, como proveedores, gestión de productos, administración de roles y permisos, etc. Estos deberán conservar su interfaz actual.

El trabajo será individual y mantendrá las reglas generales establecidas en la etapa 01.

Objetivos:

- Desarrollar una aplicación móvil utilizando Flutter y Dart.
- Construir interfaces que contemplen diferentes tamaños de pantalla y estados de interacción.
- Separar presentación, gestión del estado y acceso a servicios.
- Consumir servicios del backend mediante peticiones asincrónicas.
- Integrar autenticación y acceso a funcionalidades según el usuario.
- Implementar un proceso de compra consistente con las reglas del negocio.
- Gestionar errores de conexión, validación y procesamiento.
- Aplicar controles de seguridad tanto en Flutter como en Laravel.
- Incorporar pruebas y documentación.
- Justificar las decisiones de arquitectura e implementación.
- Aplicar validación, autenticación, autorización y controles de seguridad.
- Incorporar pruebas y documentación.
- Justificar técnicamente las decisiones adoptadas.

Requisitos de aprobación

1. **Catálogo de productos**

La aplicación deberá permitir explorar el catálogo sin iniciar sesión e incluir:

- Listado de productos con nombre, imagen y descripción breve.
- Navegación y filtrado por categorías.
- Identificación y consulta de novedades.
- Identificación y consulta de productos destacados.
- Búsqueda por nombre.
- Ordenamiento por nombre y fecha de incorporación.
- Paginado de resultados.

Para los usuarios autenticados se incorporará, además, ordenamiento por precio ascendente y descendente.
Se deberá definir y documentar el criterio utilizado para considerar un producto como novedad. Los productos destacados deberán identificarse a partir de información administrada en el backend.
Al cambiar los filtros o el ordenamiento, la aplicación deberá reiniciar el paginado y mantener coherencia entre los criterios seleccionados y los resultados mostrados.
Los visitantes no podrán consultar precios. Esta restricción deberá aplicarse tanto en la interfaz como en las respuestas del backend: los endpoints públicos no deberán devolver precios ni otros importes comerciales protegidos. En su lugar, la aplicación mostrará una indicación para iniciar sesión.

2. Detalle del producto

Al seleccionar un producto se deberá mostrar:

- Nombre.
- Imagen o galería de imágenes.
- Descripción completa.
- Categoría.
- Condición de novedad o destacado, cuando corresponda.
- Disponibilidad.
- Precio, exclusivamente para usuarios autenticados.
- Selector de cantidad y acción para agregar al carrito, exclusivamente para usuarios autenticados.

La aplicación deberá contemplar productos inexistentes, inactivos o sin disponibilidad, evitando acciones que no puedan completarse.

3. Registro y autenticación

La aplicación deberá permitir:

- Registrar un cliente con los datos mínimos necesarios.
- Iniciar sesión.
- Consultar los datos básicos del usuario autenticado.
- Cerrar sesión.

Se deberán reutilizar las cuentas y los permisos del backend, adaptando el mecanismo de autenticación para su consumo desde Flutter.
La aplicación deberá gestionar sesiones vencidas o inválidas y solicitar nuevamente el inicio de sesión cuando corresponda. Las credenciales de sesión deberán almacenarse mediante un mecanismo apropiado para la plataforma.
El visitante podrá consultar el catálogo y el detalle de los productos. Para visualizar precios, administrar el carrito y realizar compras deberá autenticarse.

4. Carrito de compras

El cliente autenticado deberá poder:

- Agregar productos.
- Consultar el contenido del carrito.
- Modificar cantidades.
- Eliminar productos.
- Vaciar el carrito.
- Visualizar precios unitarios, subtotales y total.

El carrito deberá conservarse al cerrar y volver a abrir la aplicación. Se deberá documentar dónde se almacena y cómo se asocia al usuario, evitando que otro usuario del dispositivo acceda a su contenido.
Antes de confirmar una compra, el backend deberá verificar nuevamente la existencia y disponibilidad de los productos, las cantidades solicitadas y los precios vigentes.
Los totales enviados desde Flutter no deberán considerarse una fuente confiable: el servidor deberá calcular los importes de la operación.
Si cambian los precios o la disponibilidad, la aplicación deberá informar la situación y permitir revisar el carrito antes de continuar.

5. Confirmación del pedido

Desde el carrito se deberá implementar un proceso de confirmación que permita:

- Revisar los productos y cantidades.
- Verificar los datos del cliente.

- Indicar los datos de entrega o retiro, según el alcance de la tienda.
- Consultar el importe definitivo.
- Seleccionar el medio de pago disponible.
- Confirmar la operación.

El backend deberá registrar el pedido y su detalle, vinculándolo con el cliente y con los módulos de ventas y pagos desarrollados en la etapa 01.
Cada operación deberá contar con un identificador y un estado consultable. Se deberá distinguir el estado del pedido del estado de su pago.
Se deberá definir cuándo se reserva o descuenta el stock y cómo se actúa ante una cancelación o un pago fallido.
El procesamiento deberá evitar registros parciales e inconsistencias.
Las pulsaciones repetidas o los reintentos de una misma confirmación no deberán generar pedidos duplicados.

6. Pagos

La aplicación deberá incluir un flujo de pago completo asociado al pedido.
Para la aprobación se admitirá una de las siguientes modalidades:

- Integración con una pasarela de pagos en un entorno de pruebas **(promoción)**.
- Implementación de un simulador de pagos en el backend, identificado claramente como tal.

No será necesario realizar cobros reales.

(promocion) En ambos casos se deberán contemplar, como mínimo, pagos pendientes, aprobados y rechazados.
El cliente deberá poder consultar el resultado y reintentar un pago cuando corresponda, sin duplicar el pedido.
Los intentos de pago deberán quedar registrados y asociados a la operación. El backend deberá determinar y persistir su estado; no será suficiente mostrar un mensaje de éxito o modificar una variable en Flutter.
Cuando se utilice una pasarela, el backend deberá verificar el resultado mediante el mecanismo proporcionado por el proveedor. El regreso del usuario a la aplicación no constituirá, por sí solo, una confirmación de pago.

No se deberán almacenar datos sensibles de tarjetas. Las claves privadas de integración permanecerán en el backend.

7. Consulta de compras

El cliente deberá poder consultar:

- Listado de sus pedidos.
- Fecha, identificador, importe y estado.

- Detalle de los productos adquiridos.
- Estado del pago.

Cada cliente podrá acceder únicamente a sus propias operaciones. Esta restricción deberá verificarse en Laravel, incluso si se intenta consultar directamente el identificador de un pedido ajeno.

8. Integración asincrónica con Laravel

Las funcionalidades deberán operar sobre datos del backend de la etapa 01 y los que se añadan en la etapa 02. Los datos locales de prueba (en el front) podrán utilizarse durante el desarrollo, pero no reemplazarán la integración en la entrega final.

La comunicación deberá cumplir con los siguientes criterios:

- Peticiones HTTP asincrónicas.
- Respuestas de datos en formato JSON.
- Endpoints públicos y protegidos claramente diferenciados.
- Validación y autorización en el servidor.
- Respuestas de error consistentes.
- Información de paginado suficiente para navegar los resultados.
- Dirección del servidor configurable para los distintos entornos.

La aplicación Flutter no deberá conectarse directamente a la base de datos ni depender de páginas HTML o plantillas Blade.
Se deberá documentar el contrato de los endpoints: método, ruta, autenticación, parámetros, respuestas y errores esperados.

9. Interfaz y organización del proyecto

La aplicación deberá ofrecer navegación coherente y distinguir los siguientes estados:

- Carga en curso.
- Consulta sin resultados.
- Operación completada.
- Error de validación.
- Falta de conexión o servidor no disponible.
- Sesión vencida.

Se deberán evitar envíos repetidos mientras una operación esté en curso y ofrecer opciones de recuperación cuando sea posible.

El código deberá separar, como mínimo, pantallas y componentes visuales, modelos de datos, servicios de comunicación y gestión del estado. La solución adoptada deberá justificarse.

Requisitos para promoción

- Están marcados en la sección anterior con **(promoción)**.

Presentación y Justificación

Durante la presentación se deberá:

- Explicar la evolución respecto de la etapa 01.
- Mostrar la arquitectura y la separación de responsabilidades entre Flutter y Laravel.
- Demostrar el recorrido de un visitante y el de un cliente autenticado.
- Realizar una compra desde el catálogo hasta la consulta del pedido y su pago.
- Mostrar el intercambio asincrónico de datos con el backend.
- Explicar cómo se protegen los precios y las operaciones de cada usuario.
- Demostrar un error de conexión y un pago rechazado.
- Justificar las decisiones sobre estado, persistencia del carrito y gestión del stock.
- Presentar un riesgo de seguridad detectado y su solución.
- Informar las pruebas realizadas, las dificultades y el trabajo pendiente.

Recomendación de seguimiento

- Construir una tabla que permita relacionar los componentes originales con los nuevos componentes Laravel.

Funcionalidad	Pantalla Flutter	Endpoint Laravel	Acceso	Estado
Catálogo	Pantalla de productos	Ruta implementada	Público, sin precios	Pendiente / En proceso / Finalizado
Carrito	Pantalla de carrito	Ruta, si corresponde	Cliente autenticado	Pendiente / En proceso / Finalizado
Confirmación	Pantalla de compra	Ruta implementada	Cliente autenticado	Pendiente / En proceso / Finalizado
Pagos	Pantalla de pago o resultado	Ruta implementada	Según operación	Pendiente / En proceso / Finalizado

- Hitos

  - [ ] Revisar el backend de la etapa 01 y definir el alcance y el contrato de comunicación.
  - [ ] Crear la estructura Flutter e implementar catálogo y detalle.
  - [ ] Integrar autenticación y acceso a precios.
  - [ ] Implementar carrito y confirmación del pedido.
  - [ ] Incorporar pagos y consulta de compras.
  - [ ] Completar pruebas, documentación y preparación de la demostración.